

제약 조건 (envs/main)

범위: 기본 태스크 Isaac-ConstrainedALBC-TRPO-v0(constrained_albc/envs/main/, attitude-only ALBC 정책)의 constraint 시스템. 이 태스크에는 10개의 IPO constraint가 실려 있다 — **probabilistic 5개**(binary violation-probability budget) + **average 5개**(expected-magnitude budget) — envs/main/config.py에 정의되고 algorithms/constraint_trpo.py의 Constraint-TRPO가 소비한다.

본 문서는 디스크 코드에 대해 검증한 코드 레벨 레퍼런스다. 레거시 full-DOF 변형 (envs/full_dof/, Isaac-ConstrainedALBC-Full-*-v0)은 동일한 constraint 리스트 상수를 재사용하지만 다른 태스크이며 여기서 다루지 않는다.

English version: [constraints.md](#) (SSOT).

1. 전체 구조

Constraint 시스템은 깔끔하게 분리된 3개 레이어로 구성된다: **정의(definitions)**는 각 cost가 무엇을 측정하는지 결정하고, **최적화(optimization)**는 cost budget을 어떻게 강제할지 결정하며, **배선(wiring)**은 어떤 budget과 하이퍼파라미터가 실제 run에 도달하는지 결정한다.

1. **정의 (cost 함수)**. mdp/constraints.py에 10개의 per-step cost 함수가 있다. 각각 (num_envs,) 크기의 non-negative cost 텐서를 반환한다. compute_all_costs()가 매 스텝 이들을 단일 (num_envs, K) 텐서로 쌓는다 (constraints.py:326-332).
2. **최적화 (ConstraintTRPO + IPO log-barrier)**. constraint_trpo.py는 K개의 cost budget에 대한 interior-point log-barrier를 TRPO surrogate 목적함수에 직접 접어 넣는다. 별도의 Lagrangian dual도, projection 스텝도 없다 — 단일 joint scalar가 natural-gradient 스텝을 구동한다.
3. **배선 (budget)**. config.py가 각 cost 함수를 per-step budget D_k 와 이름에 바인딩한다; 알고리즘은 이를 discounted budget $d_k = D_k / (1 - \gamma_c)$ 로 재조정하고(constraint_trpo.py:146-151) 배치의 mean discounted cost return과 비교한다.

```
cost function f_k(state) -> compute_all_costs -> costs (num_envs, K) [mdp/constraints.py]
|
v per-step cost, one column per constraint
cost GAE (cost_gamma, cost_lam) -> J_hat_C_k (mean discounted cost return, K-vector)
|
v
IPO adaptive threshold: d_k^i = max(d_k, J_hat_C_k + alpha * d_k) [constraint_trpo.py:306-308]
|
v
barrier margin_k = d_k^i - cost_surr_k # plain difference, NOT / d_k
barrier = -sum_k log(margin_k.clamp(min=1e-8)) / barrier_t [constraint_trpo.py:454-464]
|
v
surrogate L = -E[A * ratio] + barrier - entropy_bonus [constraint_trpo.py:461,480]
|
v
TRPO natural-gradient step under a pure-KL trust region (barrier curvature NOT in Fisher)
```

핵심 구조적 사실: ConstraintTRPO에는 probabilistic constraint와 average constraint를 구별하는 코드가 전혀 없다. K개 열 전부가 byte-identical한 GAE, standardization, adaptive-threshold, barrier 코드를 거친다. prob/avg 구분은 오로지 각 cost 신호가 upstream에서 어떻게 계산되는지(mdp/constraints.py)와 문서 관례(config.py)에만 존재하며, optimizer가 그것을 소비하는 방식에는 전혀 없다.

2. Constraint 분류 — probabilistic vs average

실려 있는 10개 constraint는 5 + 5로 나뉜다. 이 구분은 config.py의 리스트 순서와 인라인 주석으로만 표현된다 — 엔트리 [0:5] 위에 # --- Probabilistic (5) ---, [5:10] 위에 # --- Average (5) --- (config.py:56, config.py:62). ConstraintTermCfg에는 is_probabilistic/kind/type 필드가 **없다**; 필드는 func, params, budget, name뿐이다(constraints.py:50-57).

카테고리	Cost 신호	Budget D_k 의 의미	Feasibility 목표
Probabilistic (5)	binary indicator $\mathbb{1}[\text{violated}] \in \{0, 1\}$	최대 violation 확률	$\mathbb{E}[\mathbb{1}] \leq D_k$
Average (5)	continuous non-negative cost	최대 expected magnitude	$\mathbb{E}[\text{cost}] \leq D_k$

수학적으로 두 카테고리는 동일한 constrained-MDP 객체 $\mathbb{E}[\sum_t \gamma_c^t c_k(s_t, a_t)] \leq d_k$ 이며, per-step cost c_k 가 무엇인지만 다르다. probabilistic constraint의 c_k 는 $\{0, 1\}$ indicator이므로 discounted sum은 (γ_c discount를 제외하면) violation count가 되고, budget은 확률로 읽힌다. average constraint의 c_k 는 continuous magnitude이므로 budget은 허용된 mean level로 읽힌다.

이들을 “다르게” 취급하는 메커니즘은 optimizer 안에 없다. constraints.py:14-24의 docstring이 [0]-[4]를 probabilistic, [5]-[9]를 average로 번호를 매기며, 그 순서와 근처의 cost-함수 수학(indicator vs continuous)이 구분의 전부다. 런타임에 compute_all_costs (constraints.py:326-332)는 torch.stack([t.func(...) for t in cfg.terms], dim=-1)을 수행한다 — 10개 term 전부를 동일하게 취급한다. 짚어줄 만한 결과 하나: binary indicator들의 per-constraint cost-advantage 열은 분산이 거의 0에 가까워, standardization 스텝이 std를 min=1.0으로 floor-clamp해서 이 binary 채널이 폭주하지 않도록 막는다(Section 4 참조).

3. 10개 constraint term

모든 값은 config.py:57-69에서 그대로 가져왔다. Discounted budget $d_k = D_k / (1 - \gamma_c)$ 이며 $\gamma_c = 0.99$ 이므로 $d_k = 100 \cdot D_k$.

#	이름	함수	핵심 파라미터/임계값	Budget D_k	유형	물리적으로 제한하는 것
0	attitude	attitude_limit	limit=1.396 rad (80°)	0.01	prob	roll/pitch 크기
1	arm_torque	torque_limit	limit_nm=9.5	0.08	prob	arm joint 토크
2	arm_joint_vel	velocity_limit	limit_rad_per_s=4.189	0.02	prob	arm joint 속도
3	joint1_pos	joint1_position	limit_rad=4*pi (12.566)	0.01	prob	joint-1 절대 각도
4	cumul_yaw	cumulative_yaw	limit_rad=8*pi (25.133)	0.01	prob	누적 yaw 회전
5	thruster_util	thruster_util_cost	파라미터 없음	0.40	avg	thruster 사용권한(authority)
6	rp_rate	rp_rate_cost	soft_threshold=0.5	0.10	avg	roll/pitch 각속도
7	yaw_rate	yaw_rate_cost	soft_threshold=0.55	0.10	avg	yaw 각속도

#	이름	함수	핵심 파라미터/임계값	Budget D_k	유형	물리적으로 제한하는 것
8	rp_vel_settling	rp_vel_settling_cost	settling_threshold=0.087	0.20	avg	roll/pitch 정착(settling)
9	manipulability	manipulability_cost	w_threshold=0.3	0.05	avg	arm manip- ulability 하한

리스트 바로 위 주석 하나가 이전 수술 이력을 기록한다: thruster_rate는 제거됐고 (“entropy_coef>0와 구조적으로 불양립: 노이즈만으로도 5배 위반”) thruster_sat는 thruster_util(average, budget 0.40, “원래 형태”)로 되돌려졌다 — config.py:53-54.

3.1 Probabilistic term ([0:5])

각각 $\{0, 1\}$ indicator를 반환하며, discounted-sum budget이 violation 확률이 된다.

- **attitude** — $\mathbb{1}[|roll| > limit \vee |pitch| > limit]$, limit = 1.396 rad.
- **arm_torque** — $\mathbb{1}[\max_j |\tau_j| > 9.5 \text{ Nm}]$.
- **arm_joint_vel** — $\mathbb{1}[\max_j |\dot{q}_j| > 4.189 \text{ rad/s}]$.
- **joint1_pos** — $\mathbb{1}[|\theta_1| > 4\pi]$, 즉 joint-1 절대 각도가 $\pm 4\pi$ 레일을 넘김.
- **cumul_yaw** — $\mathbb{1}[|\theta_{yaw, cum}| > 8\pi]$, 누적(unwrapped) yaw.

3.2 Average term ([5:10])

각각 continuous non-negative cost를 반환하며, discounted-sum budget이 expected magnitude가 된다.

- **thruster_util** — thruster_utilization_cost는 **설정 가능한 임계값이 없다**(ConstraintTermCfg.params 기본값 {}에 의존, constraints.py:55). docstring(constraints.py:20)에 따르면 per-env 최대 절대 정규화 thruster state $\max_i |s_i|$ 를 반환한다. 10개 중 유일한 control-authority 채널이다.
- **rp_rate** — 0.5에서 hinge된 soft-thresholded roll/pitch 각속도 cost: cost는 soft threshold를 넘어야만 발생한다.
- **yaw_rate** — 0.55에서 hinge된 soft-thresholded yaw-rate cost.
- **rp_vel_settling** — mean roll/pitch 각속도 $(|p| + |q|)/2$ 이되, attitude error가 target의 settling_threshold=0.087 rad(5 deg) 이내일 때만 nonzero로 masking된다: transit 중(att error가 threshold보다 큼)에는 cost가 0이고 settling 중(att error가 threshold 이하)에만 active다. 이름과 구현이 일치한다(constraints.py:206-228). 단 settling_threshold가 각속도 자체가 아니라 attitude error를 게이트한다는 점은 유의할 caveat이다.
- **manipulability** — manipulability 하한 w_threshold=0.3 아래에서 hinge된다; arm의 manipulability 측정치가 하한 밑으로 떨어지면 cost가 발생한다.

3.3 실험 전용 joint1 term (미출시)

joint1-anti-drift 실험 라인(Section 7)을 위한 average cost 함수 2개가 별도로 존재하며, 실려 있는 10개에는 포함되지 않는다(constraints.py:26-29):

- **joint1_centering_cost**(arm “A”) — $\text{wrap}(\theta_1)^2$, 순간 wrapped joint-1 각도(constraints.py:245).
- **joint1_cumulative_cost**(arm “B”) — $|q_1^{\text{des}} - q_1^{\text{nom}}|$, unwrapped된 누적 command displacement(constraints.py:269, 사용: env._joint_pos_targets[:, 0] - env._nominal_joint_pos[0]).

4. ConstraintTRPO 최적화

알고리즘 본체: `algorithms/constraint_trpo.py`. ConstraintTRPO는 독립형 알고리즘이며(alias ALBC-ConstraintTRPO, `rsl_rl_ppo_cfg.py:25`), `rsl_rl.PPO`의 서브클래스가 **아니다**.

4.1 IPO log-barrier

Interior-point 목적함수(모듈 docstring 형태):

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}[A(s, a)] + \frac{1}{t} \sum_k \log(d_k^i - \hat{J}_{C_k}) \\ \text{s.t.} \quad & \text{KL}(\pi \parallel \pi_i) \leq \delta_{\max} \\ \text{where} \quad & d_k^i = \max(d_k, \hat{J}_{C_k} + \alpha d_k) \end{aligned}$$

코드에서 barrier는 update마다 다음과 같이 구성된다(`constraint_trpo.py:454-464`):

$$\begin{aligned} \text{margin}_k &= \underbrace{(d_k^i - \hat{J}_{C_k})}_{\text{barrier_base}} - \text{cost_surr}_k \\ \text{cost_surr}_k &= \frac{1}{1 - \gamma_c} \mathbb{E}[\text{ratio} \cdot \hat{A}_k^C] \\ \text{barrier} &= -\frac{1}{t} \sum_k \log(\text{margin}_k \cdot \text{clamp}(\min = 10^{-8})) \end{aligned}$$

두 가지 구현상의 사실이 중요하다. 첫째, **margin은 순수 차이일 뿐, d_k 로 정규화되지 않는다** — 파일 어디에도 $\hat{J}_C/d_k$ 나눗셈이 없다. 둘째, margin의 두 항은 동일한 **discounted-cost 스케일**에 있지만 코드에서 동일한 계수를 곱하지 않는다: `barrier_base = adaptive_d_k - mean_cost_returns`는 명시적 $1/(1 - \gamma_c)$ 를 갖지 않는 반면, `cost_surr`는 `constraint_trpo.py:462`에서 `inv_one_minus_gamma`로 스케일된다(`mean_cost_returns`와 `adaptive_d_k` 둘 다 budget 재조정 $d_k = D_k/(1 - \gamma_c)$ 를 통해 이미 discounted-return 스케일에 있음). `clamp(min=1e-8)`은 barrier가 infeasibility 경계에서/이후에도 결코 $-\infty/+\infty$ 에 문자 그대로 도달하지 못하게 막는다 — 이는 interior-point 보장을 경계 근처에서 엄밀히 강제하는 대신 조용히 완화하는 것이다.

barrier_t / barrier_alpha — 실제 런타임 값(문서 drift 경고). `barrier_t = 100.0`은 클래스 생성자 기본값(`constraint_trpo.py:64`)과 `agent cfg(rsl_rl_ppo_cfg.py:217)` 둘 다 동일 — drift 없음. **barrier_alpha는 확인된 drift가 있다:** 생성자 기본값은 `0.02`(`constraint_trpo.py:65`)이지만 실제 학습된 모든 run에 도달하는 값인 `agent cfg`는 `0.05`(`rsl_rl_ppo_cfg.py:218`)다. `RslRlConstraintTRPOAlgorithmCfg` 필드는 build 시점에 생성자 kwarg로 dispatch되므로, `cfg`가 해당 필드를 공급할 때마다 **실효값은 0.05이고, 0.02 생성자 기본값은 죽은 코드다**. 일부 이전 노트는 0.02를(또는 이미 superseded된 0.05-vs-0.02 wiki 페이지를) 보고했으나, 코드 진실은 **agent cfg에서 주입되는 0.05**다. 의심스러울 땐 클래스 시그니처가 아니라 resolve된 per-run params/`agent.yaml`을 읽을 것.

4.2 Adaptive threshold와 violations 진단 지표

Adaptive threshold는 매 update마다 현재 배치의 mean cost return으로부터 재계산된다(`constraint_trpo.py:306-308, :446`에서 호출):

$$d_k^i = \max(d_k, \hat{J}_{C_k} + \alpha d_k), \quad \hat{J}_{C_k} = (\text{mean cost return})_k \cdot \text{clamp}(\min = 0)$$

보고되는 violations 모니터링 지표는 $\hat{J}_{C_k}$ 를 adaptive threshold가 아니라 **raw budget d_k 와 비교한다**: `violations = (mean_cost_returns - self.d_k)` (`constraint_trpo.py:447`). `adaptive_d_k`는 barrier margin(:454)과 로깅되는 barrier margin(:449) 안에서만 사용된다.

4.3 Feasibility, GAE, standardization

- **Cost GAE.** Per-constraint advantage는 reward GAE와 동일한 재귀적 δ/λ -return 공식을 쓰되 `cost_gamma=0.99`, `cost_lam=0.95`, vectorized K-dim accumulator를 사용하고 시간 역순으로 실행한 뒤, rollout 전체에서 NaN/Inf를 포함하는 constraint 열을 0으로 만드는 non-finite sanitize를 거친다(`constraint_trpo.py:285-300`). `cost_gamma`는 반드시 < 1 이어야 하며 아니면 생성자가 `ValueError`를 던진다(:143-144).
- **Time-out bootstrapping.** time-out 시(termination이 아님) per-constraint cost는 `cost_gamma`와 per-constraint value 벡터로 bootstrap되며, 이는 reward 측 bootstrap을 cost 채널에서 그대로 미리링한다(`constraint_trpo.py:264-266`).
- **Per-constraint cost-advantage standardization (NORBC Sec IV-B).** K개 cost-advantage 열 각각은 flatten된 batch 차원에서 독립적으로 mean/std-정규화되며, std는 `min=1.0`으로 floor-clamp된다(`constraint_trpo.py:437-438`). 이는 통상적인 $1e-8$ epsilon이 아니라 의도적인 anti-amplification 선택이다: 인라인 주석(:436)은 `# clamp(min=1.0): binary constraints can have near-zero std, causing 1e8 amplification.`라고 적혀 있다. Reward advantage는 별도로 통상적인 $1e-8$ guard로 정규화된다(:424-426).

4.4 TRPO trust-region 스텝

Natural-gradient 스텝은 단일 결합 surrogate scalar에 대해 동작한다(`constraint_trpo.py:461,480`):

$$L = -\mathbb{E}[A \cdot \text{ratio}] + \text{barrier} - \beta_{\text{ent}} H$$

- **Gradient 그룹.** $g = \nabla_{\theta} L$, $\theta = (\text{actor}, \text{encoder}, \log \sigma)$ 에 대해.
- **Fisher-vector product**는 순수 KL curvature만 사용한다 — old policy와 new policy 사이 Gaussian KL을 통한 double backprop에 Tikhonov damping $Fv + \lambda_{\text{cg}} v$ 가 더해진다(`constraint_trpo.py:354-365`). barrier의 curvature는 Fisher에 결코 들어가지 않으므로, KL trust region은 constraint 기하학을 “알지” 못한다.
- **Conjugate gradient.** `cg_iters=10`과 조기 종료 residual tolerance $1e-10$ 으로 $Fx = g$ 를 푼다(:367-386).
- **스텝 크기.** $\Delta\theta = -\sqrt{\delta_{\text{max}}/(\frac{1}{2}\tilde{g}^{\top}g)}\tilde{g}$; $\frac{1}{2}\tilde{g}^{\top}g \leq 0$ 이거나 non-finite이면, 또는 스텝 방향에 NaN/Inf가 있으면 update가 중단된다(False 반환, 스텝 없음, :539-547).
- **Line search.** Backtracking, shrink factor 0.5, 최대 10회 backtrack; surrogate가 엄밀히 감소하고 동시에 실현된 $\text{KL} \leq \delta_{\text{max}} \cdot \text{line_search_kl_margin}$ (margin 1.5)일 때만 스텝을 수락한다(:400-410). 수락 경계가 초기 스텝 크기를 정하는 δ_{max} 보다 더 느슨하다는 점에 유의 — 수락된 스텝은 명목 trust-region 반경을 최대 50%까지 초과할 수 있다. 소진 시 파라미터는 `old_params`로 되돌아간다.

실제 trust-region 값. `max_kl`도 drift한다: 생성자 기본값 0.002 (`constraint_trpo.py:43`) vs agent cfg 0.005(`rs1_rl_ppo_cfg.py:191`) — agent cfg(0.005, 2.5배 더 느슨함)가 이긴다. `cg_damping=0.1`과 `line_search_kl_margin=1.5`는 생성자와 cfg가 일치한다(drift 없음).

4.5 std clamping

매 `_trpo_step` 이후(line-search 성공 여부와 무관하게) `log_std`가 clamp된다(`constraint_trpo.py:485-491`). 런타임에는 `min_std_per_dim`이 non-empty이므로 per-dim floor 분기가 실제로 실행되는 것이다:

$$\text{min_std_per_dim} = (0.10, 0.10, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05), \quad \text{max_std} = 2.0$$

Arm dim(인덱스 0,1)은 0.10에서, thruster dim(2-7)은 0.05에서 floor된다(`rs1_rl_ppo_cfg.py:236`). scalar `min_std=0.05` agent-cfg 필드는 `min_std_per_dim`이 non-empty일 때(기본값이 그러함) 런타임에 죽은 코드다 — per-dim 텐서만 적용된다. 생성자의 scalar 기본값 (`min_std=0.01`, `min_std_per_dim=()`)은 scalar 분기를 타겟지만, cfg가 둘 다 override한다.

Entropy bonus도 동일한 per-dim-wins 패턴을 따른다: agent-cfg entropy_coef_per_dim = (0.01, 0.01, 0.001 × 6)(rsl_rl_ppo_cfg.py:229)가 scalar entropy_coef=0.003(:223)을 override하므로, 후자도 런타임에는 죽은 코드다.

4.6 Cost critic

- **파라미터 그룹 분할.** 두 개의 서로소 그룹. **정책 그룹**(actor + encoder + log_std)은 TRPO natural gradient로 업데이트된다 — Adam도, optimizer 객체도 없음; log_std는 KL trust region이 노이즈 변화를 제한하도록 의도적으로 이 그룹에 포함되며, encoder도 분리 시 encoder 그래디언트가 ~85% 감소한다는(코드 주석, constraint_trpo.py:154-158) 이유로 이 그룹에 유지된다. **value 그룹**(value_prefixes 튜플 critic. / cost_critic. / value_backbone. / reward_head. / cost_head.에 매칭)은 value_lr로 Adam이 업데이트한다(:160-186).
- **단일 multi-head cost critic.** cost critic은 관측당 K차원 벡터를 생성하는 **하나의** 네트워크이며(evaluate_costs()), K개의 별도 네트워크가 **아니다**. Reward critic과 cost critic은 하나의 backward pass에서 함께 업데이트된다: $\text{total} = \text{value_loss_coef} \cdot L_V + \text{cost_value_loss_coef} \cdot L_{V_C}$, 두 계수 모두 1.0(constraint_trpo.py:591-605). Per-constraint MSE는 배치에 대해 .mean(dim=0)으로 K-vector를 만든 뒤, constraint에 대해 .mean()으로 scalar가 된다(:596-597). Critic head의 activation 상세는 이 파일이 아니라 정책 클래스 ALBCActorCriticEncoder에 있다 — network-architecture 레퍼런스 참조.

5. Constraint-margin 정규화 ($\hat{J}_C/d_k$)

Optimizer는 절대 margin을 저장한다; binding/slack 판단을 하려면 반드시 먼저 d_k 로 정규화해야 한다. 이는 분석 규칙이지 코드 동작이 아니다 — barrier 자체는 절대 margin을 올바르게 사용하지만(Section 4.1), 사람이나 엔진이 TensorBoard에서 Constraint/margin/<name>을 읽을 때는 constraint 간 비교를 위해 d_k 로 나눠야 한다.

부호가 뒤집히는 이유: discounted budget $d_k = 100 \cdot D_k$ 는 10개 constraint에 걸쳐 **40배**의 범위를 갖는다(attitude는 $D_k = 0.01$ 에서 $d_k = 1.0$, thruster_util은 $D_k = 0.40$ 에서 $d_k = 40.0$). Slack이 깊지만 budget이 큰 constraint는 절대 margin이 크게 나오고, budget에 가깝지만 budget이 작은 constraint는 절대 margin이 작게 나온다. 따라서 raw margin을 그대로 읽으면 결론이 뒤바뀐다.

올바른 정규화 값은 다음과 같다.

$$\frac{\hat{J}_{C_k}}{d_k} = 1 - \frac{\text{margin}_k}{d_k},$$

이는 adaptive floor가 작동하지 않은, 즉 $\text{Constraint/viol/<name>} = -\text{Constraint/margin/<name>}$ 이 정확히 성립하는 **slack 영역에서만** 유효하다(constraint_trpo.py:447 vs :449). $\hat{J}_{C_k} + \alpha d_k > d_k$ 이면 adaptive threshold가 작동해 $d_k^i \neq d_k$ 가 되고, 그 identity가 깨진다.

실험적 발견(그렇게 명시). 실제 teacher-run 리포트(report.md:174,180)가 attitude와 cumul_yaw를 절대 margin이 숫자상 작아 보인다는 이유로 “binding family”로 오독한 사례가 있다 — 실제로는 $\hat{J}_C/d_k = 0.003$ 과 0.000 (가장 깊은 slack)이었던 반면, 진짜로 binding인 thruster_util ($\hat{J}_C/d_k \approx 0.87$)은 단지 budget이 커서 절대 margin이 크게 나온 것이었다. 분석 엔진에는 아직 이 gap이 남아 있다: analyze_training.py:_constraint_margin()(:370-379)은 d_k 정규화 없이 절대 margin을 반환한다. 출처: omx wiki constraint_margin_must_be_normalized_j_c_d_k_absolute_margin_fli.md.

6. DORAEMON과 constraint

DORAEMON은 domain-randomization 커리큘럼이다. 여기서 다루는 초점은 constraint/feasibility 메커니즘과의 상호작용이며, DR 메커니즘 자체는 marinelab에 있다. 메인 env는 네 필드를 override한다(config.py:479): DoraemonCfg(enable=True, kl_ub=0.12, performance_lb=250.0, step_interval=250).

- **alpha는 feasibility floor이지, DR lever가 아니다.** DORAEMON의 alpha(config 근거 주석에서 0.5로 인용됨, config.py:473-474; 실제 필드 기본값은 marinelab의 DoraemonCfg에서 오며 읽은 파일들에서는 검증되지 않음)는 curriculum-difficulty 스텝이 수락되는지 여부를 게이트한다 — randomization을 직접 넓히지 않는다. **실험적 발견:** alpha를 0.50 → 0.75로 올린 것(E5, run trpo_260606_225859)은 거의 null intervention이었다 — success_rate가 run 내내 0.96-0.99로 포화돼 있어 floor가 스텝을 게이트한 적이 없었기 때문이다. DR-expansion 속도를 실제로 제약하는 것은 per-update trust-region KL($k1_ub$, 그 run에서 0.06 고정)이다. robustness를 넓히려면 alpha가 아니라 $k1_ub$ 또는 DR 분산을 직접 움직여야 한다. 출처: omx wiki doraemon_alpha_is_a_feasibility_floor_not_a_dr_expansion_lever_e.md.
- **performance_lb와 $k1_ub$ 는 설계상 함께 올려졌다**(config.py:466-478). performance_lb는 68.0 → 250.0으로, $k1_ub$ 는 0.06 → 0.12로 올라갔다. 이 보정은 recon run(trpo_baseline_260608_160453, 1146 iter)을 사용했다: 그 run의 DORAEMON episode-return 분포는 min=81.9 / p5=227 / p25=250 / median=264 / p95=291이었다. lb=68이 최소값보다 아래에 있으면 success = return >= 68이 항상 1이 되어, feasibility constraint ($\hat{G} \geq \alpha$)가 **비활성(inert)**이 되고 curriculum이 self-pacing 피드백 없이 DR을 무제한으로 넓혔다. lb=250(p25)은 시작 success_rate를 ~0.65로 만든다 — alpha=0.5보다 위라서 분포는 여전히 확장되지만 신호는 1에 고정되지 않고 다시 살아난다; median보다 낮게 선택한 이유는 reward plateau가 success_rate를 0으로 끌어내리지 않게 하기 위함이다. $k1_ub$ 는 올라간 lb가 야기하는 느려진 확장 속도를 보상할 만큼 분포를 빠르게 넓히기 위해 두 배로 늘어났다. 두 lever는 설계상 함께 움직인다: lb 단독은 DR을 쉽게 만들고, $k1_ub$ 단독은 success_rate를 1에 고정된 채로 남긴다.
- **success의 정의.** success = accumulated_episode_return >= performance_lb, albc_env.py에서 계산됨(episode_return_accum += reward; success = return >= performance_lb; config.py:467 주석).

step_interval = 250은 curriculum이 얼마나 자주 업데이트되는지를 정한다.

7. joint1 anti-drift 실험

기본 비활성 토글이 joint-1 anti-drift를 테스트하기 위해 **11번째** constraint term을 추가할 수 있다. ALBCEnvCfg의 필드 2개:

- joint1_constraint_arm: str = "none" — {"none", "A", "B"} 중 하나 (config.py:532).
- joint1_constraint_budget: float = 0.05 — arm A/B의 per-step average budget d_k (config.py:533).

Materialization 경로. apply_joint1_constraint_arm() (constraints.py:301-323)이 arm을 읽는다; "none"은 no-op; "A"는 joint1_centering_cost term을 추가(이름 joint1_centering); "B"는 joint1_cumulative_cost term을 추가(이름 joint1_cumulative); 그 외 값은 ValueError를 던진다. env_cfg.constraints.terms = [*env_cfg.constraints.terms, term]을 통해 추가한다.

왜 cfg의 __post_init__이 아니라 ALBCEnv.__init__에서 호출되는가. 호출 지점은 albc_env.py:128, ALBCEnv.__init__ 내부(payload-viz와 DR-sampler init 이후)다. 이는 의도적이다: hydra의 update_class_from_dict override는 cfg __post_init__ 이후, ALBCEnv.__init__ 실행 이전에 적용된다(constraints.py:304-307, albc_env.py:123-127). 이 호출을 __post_init__에 넣으면 항상 arm="none"을 관측하게 되어, arm A/B 대신 조용히 baseline과 중복되는 run이 생성될 것이다.

단일 메커니즘 원칙. A/B에 대해서는 reward의 k_joint1_center 를 0.0으로 설정해서 centering 메커니즘이 정확히 하나만 활성화되게 해야 한다 (config.py:520-524). 이는 문서화된 실험자 요구사항이다 — config.py의 ALBCEnvCfg.reward 호출이 이를 자동으로 설정하지는 않는다(ALBCRewardCfg 기본값은 이미 0.0이지만, reward를 override할 때 이를 강제하는 장치는 없다).

동작 — station-keeping 하에서는 결코 binding되지 않음(실험적 발견). 평탄한 target station-keeping 태스크에서 joint1-cumulative constraint는 사실상 결코 binding되지 않는다: 정책은 $\pm 4\pi$ (2회전) 레일 깊숙이 안쪽인 **0.36 회전** 근처에 자연스럽게 자리 잡으며, violation 지표는 매 iteration **-0.9997** (거의 완전한 headroom)에 머물고, 4개 DR 난이도 레벨 전체에서 peak $|\theta_{cum}|$ 가 ~1.22 회전을 넘은 적이 없다 (64개 env 중 0개가 > 4π). 출처: omx wiki joint1_cumulative_rotation_constraint_never_binds_policy_parks_a.md (run trpo_joint1_cumul_rot_260629_183545). 주의: 그 wiki의 run은 이 term을 probabilistic $\mathbb{1}[|\theta_{cum}| >$

$4\pi] \leq 0.01$ constraint로 프레이밍하는 반면, 실려 있는 코드의 arm-“B” 함수 `joint1_cumulative_cost`는 $\text{average } |q_1^{\text{des}} - q_1^{\text{nom}}|$ cost다 — 동일한 실험 라인이 run에 따라 두 변형을 오간다; 실질(결코 binding되지 않음)은 유지된다.

Binding되지 않으면서도 OOD로 일반화됨(실험적 발견). station-keeping 하에서 결코 binding되지 않음에도, 동일한 constraint 라인은 out-of-distribution에서도 누적 회전 drift를 bounded 상태로 유지한다(drift slope 2.2×10^{-4} rad/s, p95 최종 $|\text{drift}| = 0.177 \text{ rad} < 0.224 \text{ rad budget}$) — 심지어 무관한 attitude tracker가 진짜 OOD heavy tail을 보이는 동안에도 그렇다(roll steady-state error env-median 0.36° vs env-mean 3.87° , worst env 63°). 출처: `omx wiki joint1_cumulative_ipo_constraint_generalizes_drift_bounded_at_oo.md` (run `trpo_cumul_constraint_260627_231709`). 주의: 이 페이지는 이를 “binding average-constraint”라고 부르는 반면 위의 동반 페이지는 동일 라인을 결코 binding되지 않는다고 문서화한다 — 이 불일치는 소스 지식 자체에 있으며(아마도 “binding”이 “active/present”를 느슨하게 의미하도록 쓰인 것으로 보임), 여기서는 조정하지 않고 그대로 표시해 둔다.

기본 비활성 = byte-identical. 기본값인 `arm="none"`에서는 `apply_joint1_constraint_arm`이 즉시 반환하고, 실려 있는 10개 term 집합은 그대로 유지되며 K는 10을 유지한다. A/B를 활성화하면 정확히 하나의 cost-critic head가 추가된다; MLP 레이어 수, 차원, activation은 변하지 않는다.

8. 로깅 / 지표

Constraint 지표는 학습 iteration마다 한 번 `ConstraintEncoderRunner._log_constraint_metrics`가 방출한다(override된 `log()`에서 호출되며 `self._should_log`에 게이트됨; `constraint_encoder_runner.py:248-250`). 알고리즘은 이 로깅에서만 읽히는 per-step running state(`_last_violations`, `_last_barrier_margins`, `_last_barrier_penalty`)를 유지한다(`constraint_trpo.py:129-133`).

지표	의미
Constraint/viol/<name>	$\hat{J}_{C_k} - d_k$, raw discounted budget과 비교(<code>constraint_trpo.py:447</code>); 음수 = slack
Constraint/margin/<name>	$d_k^i - \hat{J}_{C_k}$ (adaptive-threshold margin, :449); 절대값 , d_k -정규화 안 됨
Constraint/barrier_penalty	집계된 scalar barrier penalty (per-constraint 접미사 없음)
Policy/line_search_success	이번 update의 line-search 수락을
Policy/entropy	평균 정책 entropy

<name> 접미사는 constraint에 설정된 이름이며 (`ALBCConstraintCfg.constraint_names`, `constraints.py:77-78`), 없으면 숫자 인덱스로 대체된다. 네임스페이스는 `viol/margin`에 대해 2-레벨 `Constraint/<type>/<name>` 계층과 flat한 `Constraint/barrier_penalty`로 구성된다; DORAEMON curriculum 지표는 별도의 `DORAEMON/*` 네임스페이스를, 정책 진단은 `Policy/*` 네임스페이스를 쓴다.

Anomaly threshold는 분석 엔진이 플래그한다(`analyze_training.py ANOMALY_RULES`): `line_search_success < 0.5 FAIL`(“TRPO line search failing. Cost gradient may dominate. Check barrier_t and constraint budgets.”), `barrier_penalty > 0.1 SPIKE`; 그리고 일반 RL-health 규칙 `entropy < 0 COLLAPSED`, `noise_std < 0.25 LOW` / `>= 0.95 CEILING`, `z_std < 0.1 LOW`, `grad_norm < 1e-4 DEAD`, `roll_deg > 20 HIGH`, `pitch_deg > 25 HIGH`. **오래된 wiki 노트에 대한 정정:** encoder-latent saturation 규칙은 실제 엔진 코드에서 `Encoder/z_min < -0.98` / `Encoder/z_max > 0.98`을 쓰며(`analyze_training.py:41-42`), 일부 노트가 말하는 ± 0.95 가 **아니다**. `thruster_util_max > 0.95`는 wiki에서 saturation anomaly로 인용되지만 엔진 코드의 `ANOMALY_RULES`에는 대응 항목이 없다 — 코드로 인코딩된 규칙이 아니라 분석 heuristic으로 취급할 것.

9. 실제로 binding되는 constraint (실험적 발견)

이 섹션의 모든 내용은 이전 분석 run에서 나온 실험 결과이며, 위의 코드 정의와는 의도적으로 분리해 두었다. 이것들을 코드 불변식으로 취급하지 말 것; 특정 run이 보여준 것일 뿐이다.

thruster_util만 binding된다. 실험에 있는 10개 constraint 중 thruster_util만이 실제로 discounted budget에 도달한다 — teacher run에서 $\hat{J}_C/d_k \approx 0.869-0.870$. 나머지 9개는 slack에 있으며, 여러 개는 상당히 깊은 slack이다: rp_vel_settling 0.455, arm_torque 0.407, rp_rate 0.319, yaw_rate 0.138(slack); manipulability 0.038, arm_joint_vel 0.031, joint1_pos 0.005, attitude 0.003(deep slack); cumul_yaw 0.000(“완전히 inert”). 출처: omx wiki constraint_margin_must_be_normalized_j_c_d_k_absolute_margin_fli.md. (참고: teacher run의 binding 수치는 ~0.87이며, 이 teacher run에서 0.94라는 근거는 없다 — 아래 0.944 수치는 teacher가 아니라 E6 budget-halving run의 것이다.)

Budget $\times 0.5$ 는 authority를 고갈시킨다. E6 실험은 10개 budget 전부를 반으로 줄였다. thruster_util만 반응해서 binding으로 더 파고들었다 ($\hat{J}_C/d_k$ 가 0.869 $\rightarrow$ 0.944로 상승); 나머지 9개는 slack에 머물렀다. control-authority 채널에서의 그 추가 binding 하나가 authority starvation을 일으켰다: per-step reward가 54% 하락했고(Reward/total 7.96 $\rightarrow$ 3.68), lin_vel reward가 음수가 됐으며, 정책 entropy가 붕괴했다(iter 2289에서 0을 통과, teacher run에서는 본 적 없는 anomaly). 규칙: control-authority 채널(thruster)을 조이는 것은 파괴적이다. 출처: omx wiki constraint_budget_x0_5_binds_only_thruster_util_authority_starva.md. 이 binding 결론은 slack 영역에서 계산된 것이므로 ($\hat{J}_C = d_k - \text{margin}$) barrier_alpha와 무관하며 0.02, 0.05 어느 쪽에서도 성립한다.

success = return ≥ 68 이었을 때 feasibility constraint는 inert했다. 과거 performance_lb=68이 최소 episode return보다 아래에 있어서 success가 1에 고정됐고, DORAEMON feasibility constraint가 전혀 발동하지 않았으며, curriculum이 self-pacing 피드백 없이 DR을 넓혔다 — Section 6에서 lb 68 $\rightarrow$ 250 / kl_ub 0.06 $\rightarrow$ 0.12 재보정의 정확한 동기다.

소스 파일

- constrained_albc/envs/main/mdp/constraints.py — 10개 cost 함수, ConstraintTermCfg, ALBC-ConstraintCfg, compute_all_costs, apply_joint1_constraint_arm
- constrained_albc/envs/main/config.py — ALBCEnvCfg, _FULL_DOF_CONSTRAINT_TERMS(실험에 있는 10개 budget), DORAEMON override, joint1 토클
- constrained_albc/envs/main/config_noconstraint.py — ALBCNoConstraintEnvCfg(terms=[], TRPO-NoIPO / PPO-Enc ablation)
- constrained_albc/envs/main/algorithms/constraint_trpo.py — ConstraintTRPO + IPO barrier, adaptive threshold, cost GAE, TRPO step, std clamp, cost critic
- constrained_albc/envs/main/agents/rs1_rl_ppo_cfg.py — Rs1RLConstraintTRPOAlgorithmCfg(런타임 barrier_alpha/max_kl/std/entropy 값)
- constrained_albc/envs/main/runners/constraint_encoder_runner.py — _log_constraint_metrics, num_constraints auto-sync
- .omx/profile/analyze_training.py — ANOMALY_RULES, _constraint_margin